

Файл: .env.py

```
python
# Секретный ключ для подписи JWT токенов
# Внимание: в реальных проектах такие ключи должны храниться в защищённом хранилище с
# екретов
JWT_SECRET=dgdfgdfg345tdhfds**/sg
```

Файл: app.py (auth_service)

```
python
# Импорт модуля Flask для создания веб-приложения
from flask import Flask, request, jsonify

# Импорт функции для загрузки переменных окружения из файла .env
from dotenv import load_dotenv

# Импорт библиотеки для работы с JWT токенами
import jwt

# Импорт модуля для работы с датой и временем
import datetime

# Импорт модуля для работы с переменными окружения операционной системы
import os

# Импорт декоратора wraps для сохранения метаданных функций (имя, докстринг и т.д.)
from functools import wraps

# Загрузка переменных окружения из файла .env в переменные окружения процесса
load_dotenv()

# Создание экземпляра Flask приложения - это главный объект приложения
app = Flask(__name__)

# Получение секретного ключа из переменной окружения JWT_SECRET
# Этот ключ используется для подписи и верификации JWT токенов
SECRET_KEY = os.getenv('JWT_SECRET')

# Проверка: если секретный ключ не установлен, используем значение по умолчанию
# В Docker переменная передаётся через docker-compose.yml
if not SECRET_KEY:
```

```

SECRET_KEY = "default_secret_key_for_docker_testing"

# Словарь с демо-пользователями для тестирования
# В реальном приложении данные должны храниться в базе данных с хэшированными паролям
и
USERS = {
    "admin": "admin123",
    "user": "user123"
}

# Декоратор для проверки валидности JWT токена
# Декоратор оборачивает функции маршрутов, требующих аутентификации
def token_required(f):
    # @wraps сохраняет метаданные оригинальной функции (имя, документацию)
    @wraps(f)
    def decorated(*args, **kwargs):
        token = None
        # Извлечение токена из заголовка Authorization
        # Ожидается формат: "Bearer <token>"
        if 'Authorization' in request.headers:
            # Разделяем строку по пробелу и берём вторую часть (сам токен)
            token = request.headers['Authorization'].split(" ")[1]

        # Если токен отсутствует - возвращаем ошибку 401 Unauthorized
        if not token:
            return jsonify({'message': 'Токен отсутствует!'}), 401

        try:
            # Декодируем и верифицируем токен с использованием секретного ключа
            # Алгоритм HS256 (HMAC-SHA256) - симметричное шифрование
            data = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
            # Извлекаем имя пользователя из payload токена
            current_user = data['username']
        except jwt.ExpiredSignatureError:
            # Ошибка: срок действия токена истёк
            return jsonify({'message': 'Токен истёк!'}), 401
        except jwt.InvalidTokenError:
            # Ошибка: токен недействителен (неправильная подпись, повреждён и т.д.)
            return jsonify({'message': 'Неверный токен!'}), 401

        # Вызываем оригинальную функцию, передавая ей current_user как первый аргумен
        return f(current_user, *args, **kwargs)
    return decorated

```

```

# Маршрут для логина (аутентификации)
# Метод POST, ожидает JSON с username и password
@app.route('/login', methods=['POST'])
def login():
    # Получаем JSON данные из тела запроса
    data = request.get_json()

    # Валидация: проверяем наличие обязательных полей
    if not data or not data.get('username') or not data.get('password'):
        return jsonify({'message': 'Необходимо указать username и password'}), 400

    username = data['username']
    password = data['password']

    # Проверяем учётные данные (в реальном приложении проверяли бы хэш пароля)
    if username in USERS and USERS[username] == password:
        # Генерируем JWT токен
        token = jwt.encode({
            'username': username, # Полезная нагрузка (payload) - данные пользовател
            'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1) # Время
            истечения (1 час)
        }, SECRET_KEY, algorithm="HS256")

        # Возвращаем токен клиенту
        return jsonify({
            'message': 'Успешная авторизация',
            'token': token, # JWT токен для последующих запросов
            'expires_in': 3600 # Время жизни в секундах (1 час)
        }), 200
    else:
        # Неверные учётные данные
        return jsonify({'message': 'Неверные учетные данные'}), 401

# Маршрут для проверки токена
# Метод POST, требует валидный токен в заголовке Authorization
@app.route('/verify', methods=['POST'])
@token_required # Применяем декоратор аутентификации
def verify_token(current_user):
    # Если декоратор пропустил выполнение - токен валиден
    return jsonify({
        'message': 'Токен валиден',
        'user': current_user
    })

```

```

    }), 200

# Маршрут для health check (нужен для Docker)
# Используется для проверки работоспособности контейнера
@app.route('/health', methods=['GET'])
def health():
    # Возвращает статус healthy если приложение работает
    return jsonify({'status': 'healthy'}), 200

# Точка входа приложения
if __name__ == '__main__':
    # Запуск Flask сервера
    # host='0.0.0.0' - слушаем все сетевые интерфейсы (нужно для Docker)
    # port=5001 - порт приложения
    # debug=False - отключаем режим отладки в production
    app.run(host='0.0.0.0', port=5001, debug=False)

```

Файл: requirements.txt

```

txt
# Flask - веб-фреймворк для создания приложения
Flask==3.0.0
# PyJWT - библиотека для работы с JWT токенами (генерация и верификация)
PyJWT==2.8.0
# python-dotenv - загрузка переменных окружения из .env файла
python-dotenv==1.0.0
# Werkzeug - набор утилит для WSGI (используется Flask)
Werkzeug==3.0.1
# gunicorn - WSGI HTTP сервер для production окружения
gunicorn==21.2.0

```

Файл: .env (1).py (data_service)

```

python
# Секретный ключ для верификации JWT токенов
# ДОЛЖЕН совпадать с ключом в auth_service, иначе верификация не пройдет
JWT_SECRET=202b7cbbd91dc75d851e6fd1045e77d3e2475c034868baaa4a78fe638fba7a7d

```

Файл: app (1).py (data_service)

```

python
# Импорт модуля Flask для создания веб-приложения
from flask import Flask, request, jsonify

```

```

# Импорт функции для загрузки переменных окружения из файла .env
from dotenv import load_dotenv

# Импорт библиотеки для работы с JWT токенами
import jwt

# Импорт модуля для работы с переменными окружения операционной системы
import os

# Импорт декоратора wraps для сохранения метаданных функций
from functools import wraps

# Загрузка переменных окружения из файла .env
load_dotenv()

# Создание экземпляра Flask приложения
app = Flask(__name__)

# Получение секретного ключа из переменной окружения JWT_SECRET
# Этот ключ используется для верификации токенов, приходящих от клиента
SECRET_KEY = os.getenv('JWT_SECRET')

# Проверка: если секретный ключ не установлен, используем значение по умолчанию
if not SECRET_KEY:
    SECRET_KEY = "default_secret_key_for_docker_testing"

# Декоратор для проверки валидности JWT токена
# Аналогичен декоратору в auth_service
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = None
        # Извлекаем токен из заголовка Authorization
        if 'Authorization' in request.headers:
            token = request.headers['Authorization'].split(" ")[1]
        if not token:
            return jsonify({'message': 'Токен отсутствует!'}), 401
        try:
            # Верифицируем токен с тем же секретным ключом
            data = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
            current_user = data['username']
        except jwt.ExpiredSignatureError:
            return jsonify({'message': 'Токен истёк!'}), 401

```

```

        except jwt.InvalidTokenError:
            return jsonify({'message': 'Неверный токен!'}), 401
        return f(current_user, *args, **kwargs)
    return decorated

# Маршрут для получения защищённых данных
# Метод GET, требует валидный JWT токен
@app.route('/protected-data', methods=['GET'])
@token_required # Любой аутентифицированный пользователь может получить эти данные
def get_protected_data(current_user):
    # Возвращаем защищённые данные, доступные любому авторизованному пользователю
    return jsonify({
        'message': 'Доступ разрешён',
        'user': current_user, # Показываем, какой пользователь сделал запрос
        'data': {
            'secret_info': 'Это защищённые данные', # Пример защищённых данных
            'timestamp': '2024-01-01T12:00:00Z'
        }
    }), 200

# Маршрут только для администраторов
# Метод GET, требует валидный JWT токен И роль администратора
@app.route('/admin-only', methods=['GET'])
@token_required # Сначала проверяем аутентификацию
def admin_only(current_user):
    # Затем проверяем авторизацию (роль пользователя)
    if current_user != 'admin':
        # 403 Forbidden - пользователь аутентифицирован, но не имеет прав
        return jsonify({'message': 'Доступ запрещён. Требуется роль администратора'})
    , 403

    # Если пользователь admin - возвращаем административные данные
    return jsonify({
        'message': 'Доступ администратора разрешён',
        'admin_data': {
            'users_count': 100, # Пример статистики
            'system_status': 'operational' # Статус системы
        }
    }), 200

# Маршрут для health check (нужен для Docker)
# Используется для проверки работоспособности контейнера
@app.route('/health', methods=['GET'])
def health():

```

```

    return jsonify({'status': 'healthy'}), 200

# Точка входа приложения
if __name__ == '__main__':
    # Запуск Flask сервера на порту 5002 (отличается от auth_service)
    app.run(host='0.0.0.0', port=5002, debug=False)

```

Файл: docker-compose.yml

```

yaml
# Версия формата docker-compose
# version 3.8 поддерживает большинство современных функций Docker
version: '3.8'

# Определение сервисов (контейнеров), которые будут запущены
services:
    # Auth сервис (генерация и верификация токенов)
    auth-service:
        # Сборка образа из папки auth_service
        build:
            context: ./auth_service # Путь к директории с Dockerfile и кодом
            dockerfile: Dockerfile # Имя Dockerfile (по умолчанию, но можно указать явно)

        # Имя контейнера (удобно для обращения из других контейнеров)
        container_name: jwt-auth-service

        # Мappings портов (хост:контейнер)
        # Пробрасываем порт 5001 контейнера на порт 5001 хоста
        ports:
            - "5001:5001"

        # Переменные окружения, передаваемые в контейнер
        environment:
            # Секретный ключ из переменной окружения хоста (${JWT_SECRET} подставляется из
            .env)
            - JWT_SECRET=${JWT_SECRET}
            # Режим Flask (development/production)
            - FLASK_ENV=${FLASK_ENV}

        # Сеть для связи между сервисами
        networks:
            - jwt-network # Используем сеть jwt-network

```

Политика перезапуска: unless-stopped - перезапускать всегда, кроме явной останова
вки

restart: unless-stopped

Health check - проверка работоспособности сервиса

healthcheck:

test: ["CMD", "curl", "-f", "http://localhost:5001/health"] *# Команда проверки*

interval: 30s *# Проверять каждые 30 секунд*

timeout: 10s *# Таймаут выполнения команды*

retries: 3 *# Количество попыток до пометки как unhealthy*

Data сервис (защищённые данные)

data-service:

Сборка образа из nanpi data_service

build:

context: ./data_service

dockerfile: Dockerfile

Имя контейнера

container_name: jwt-data-service

Мappings портов (хост:контейнер)

ports:

- "5002:5002" *# Пробрасываем порт 5002 контейнера на хост*

Переменные окружения

environment:

Секретный ключ из .env файла (должен совпадать с auth-service!)

- JWT_SECRET=\${JWT_SECRET}

Режим Flask

- FLASK_ENV=\${FLASK_ENV}

Сеть для связи между сервисами

networks:

- jwt-network

Зависимость от auth-service

Docker Compose запускает auth-service перед data-service

depends_on:

- auth-service

Политика перезапуска

restart: unless-stopped


```
# Health check для проверки здоровья data-service
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:5002/health"]
  interval: 30s
  timeout: 10s
  retries: 3

# Определение сетей
networks:
  # Изолированная сеть для сервисов
  jwt-network:
    driver: bridge # Используем драйвер bridge (стандартная сеть для контейнеров)
```

Дополнительные комментарии по архитектуре:

1. Разделение сервисов:

- `auth-service` отвечает только за аутентификацию и выдачу токенов
- `data-service` содержит бизнес-логику и защищённые данные
- Такое разделение позволяет масштабировать сервисы независимо

2. Безопасность:

- Секретный ключ должен быть одинаковым для обоих сервисов
- В реальном проекте ключ должен храниться в защищённом хранилище (например, HashiCorp Vault)
- Токены имеют ограниченное время жизни (1 час)

3. Docker особенности:

- Используется общая сеть `jwt-network` для взаимодействия сервисов
- Health checks помогают оркестратору отслеживать состояние сервисов
- `depends_on` гарантирует порядок запуска, но не проверяет готовность сервиса

ПОШАГОВЫЙ АЛГОРИТМ

◆ Шаг 1: Проверьте работу сервисов

Откройте браузер или новую вкладку Postman:

Метод	URL	Ожидаемый ответ 
GET	<code>http://localhost:5001/health</code>	<code>{"status": "healthy"}</code>
GET	<code>http://localhost:5002/health</code>	<code>{"status": "healthy"}</code>

 

```
1
2   Если оба возвращают "healthy"
3    → переходите к Шагу 2
4
5   Если ошибка → проверьте Docker:
6    docker-compose ps
7
```

Запрос 1: Login

Метод: `POST`

URL: `http://localhost:5001/login`

Headers:

 

```
1 Content-Type: application/json
```

Body (raw, JSON):

json  

```
1 {
2   "username": "admin",
3   "password": "admin123"
4 }
```

Ожидаемый ответ:

```
json
1 {
2   "message": "Успешная авторизация",
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
4   "expires_in": 3600
5 }
```

Запрос 2: Protected Data

Метод: GET

URL: <http://localhost:5002/protected-data>

Headers:

```
1 Authorization: Bearer {{jwt_token}}
```

Ожидаемый ответ:

```
json
1 {
2   "message": "Доступ разрешён",
3   "user": "admin",
4   "data": {
5     "secret_info": "Это защищённые данные",
6     "timestamp": "2024-01-01T12:00:00Z"
  }
```

Запрос 3: Admin Only

Метод: GET

URL: <http://localhost:5002/admin-only>

Headers:

```
1 Authorization: Bearer {{jwt_token}}
```

Ожидаемый ответ (admin):

json

```
1 {  
2   "message": "Доступ администратора разрешён",  
3   "admin_data": {  
4     "users_count": 100,  
5     "system_status": "operational"  
6   }  
7 }
```

13. Диагностика проблем

Проблема 1: Контейнеры не запускаются

bash

```
1 # Проверка логов
2 docker-compose logs auth-service
3 docker-compose logs data-service
4
5 # Проверка портов
6 netstat -an | findstr "5001"
7 netstat -an | findstr "5002"
```

Проблема 2: Ошибка JWT_SECRET

bash

```
1 # Проверка переменных в контейнере
2 docker exec jwt-auth-service env | findstr JWT_SECRET
```

Проблема 3: Health check fails

Проблема 2: Ошибка JWT_SECRET

bash

- 1 # Проверка переменных в контейнере
- 2 docker exec jwt-auth-service env | findstr JWT_SECRET

Проблема 3: Health check fails

bash

- 1 # Проверка внутри контейнера
- 2 docker exec jwt-auth-service curl <http://localhost:5001/health>

Проблема 4: Пересборка образов

bash

- 1 # Принудительная пересборка
- 2 docker-compose build --no-cache
- 3 docker-compose up -d